

W8 Handout — DevOps + CI/CD

225381 · Application Development with Cloud Platform · Week 8

Overview

This week closes the loop between W1–W7 (theory) and W13–W14 (Azure hands-on). The lab is your first end-to-end CI/CD pipeline: push code to GitHub, watch tests run automatically, see a public URL update. Estimated self-study: 2 hours.

Learning objectives:

1. Explain DevOps as a culture, not just a toolset (CALMS framework)
2. Differentiate Agile, DevOps, and DevSecOps — what each added
3. Distinguish CI, Continuous Delivery, and Continuous Deployment
4. Describe the 5 stages of a typical pipeline (commit → build → test → deploy → monitor)
5. Explain why cloud-native apps (microservices + containers) make CI/CD easier
6. Identify the 5 common challenges to CI/CD adoption
7. Build a working GitHub Actions pipeline for a small Node.js or Python app

1. What is DevOps? (Ch 11, pp. 149–155)

DevOps is the combination of **Development** and **Operations** — an approach that emphasizes collaboration to deliver software rapidly, reliably, and repeatedly. It is a **culture** enabled by tools, not a set of tools by itself.

The CALMS framework

Letter	Pillar	What it means in practice
C	Culture	Shared responsibility between dev and ops — no "us vs them"
A	Automation	Build, test, deploy, monitor — scripts, not humans
L	Lean	Small batches, fast feedback, eliminate waste
M	Measurement	You can't improve what you don't measure (deploy time, MTTR, change-failure rate)
S	Sharing	Devs and ops share tools, dashboards, on-call rotations

Why this matters: Ch 11 p. 155 — "DevOps is a culture. Tools enable it, but don't create it." Buying Jenkins does not make you a DevOps team.

2. Agile → DevOps → DevSecOps — the evolution (Ch 11, p. 150)

Same goal: ship faster with higher quality. Each step removed a bottleneck.

Era	What was added	How it works	Who collaborates
Agile · ~2001	Iterative dev (Scrum, Kanban)	Plan small, build small, demo small	Dev team only
DevOps · ~2009+	Automation + ops collab	Build + deploy + monitor continuously	Dev + Ops

DevSecOps · ~2015+	Security into the loop	Scan + test + gate every pipeline run	Dev + Ops + Sec
--------------------	------------------------	---------------------------------------	-----------------

Ch 11 p. 150: "DevSecOps is the process of integrating security into the software development process." Security stops being a "last step before release" and becomes a gate in every commit.

3. CI vs Continuous Delivery vs Continuous Deployment (Ch 11, pp. 152–154)

These three terms are **NOT** the same. Both "CD"s are different things.

Stage	Continuous Integration (CI)	Continuous Delivery (CD)	Continuous Deployment (CD)
Trigger	Every code commit	Every successful CI run	Every successful test
Builds?	Yes — compile, unit test	Yes — full test suite	Yes — full test + package
Deploys to	Build server (artifact)	Staging / pre-prod	Production — automatically
Human gate?	No (automatic)	Yes — manual approval to prod	No (fully automatic)
Risk	Low	Medium	Medium-high
When to use	Always (start here)	When you need prod control	When you trust your tests fully

Memory aid: CD has two meanings. **Continuous Delivery** = human-approved deploy. **Continuous Deployment** = automatic to prod. This is the #1 CI/CD interview question.

4. The 5-stage pipeline

Every modern pipeline has the same backbone. The tools change (Jenkins, GitHub Actions, GitLab CI, Azure Pipelines), the structure doesn't.

#	Stage	What runs	Why it matters
1	Commit	<code>git push</code>	Code enters the pipeline
2	Build	Compile, bundle, create artifact	Turn source into a runnable thing (jar, image, zip)
3	Test	Lint, unit, integration, security	Catch bugs early, block bad code from progressing
4	Deploy	Push to staging / prod	Release to users (auto or human-gated)
5	Monitor	Metrics, logs, alerts, feedback	Drives the next change

The feedback loop: stage 5 sends data back to stage 1. If the new release crashes, monitoring catches it in minutes, the team writes a fix in stage 1, the pipeline ships it, and the system heals itself.

5. Cloud native = microservices + containers + orchestration (Ch 11, p. 156)

CI/CD works best when the app is designed for the cloud from day one. The three building blocks (you've seen them in W2, W4, W5):

Building block	What it is	Why it helps CI/CD
Microservices	App split into small services	Small change → small risk → ship many per day

Containers (Docker)	App + dependencies in one image	Same artifact runs identically on dev, CI, prod
Orchestration (K8s, Container Apps)	Schedules, scales, heals containers	Auto-rollback, auto-scale, zero-downtime deploys

Why microservices enable DevOps (and vice versa):

- One team owns one service → less cross-team coordination
- If a service breaks, the blast radius is small
- Different services can be deployed at different cadences

6. Opinionated continuous delivery (Ch 11, pp. 153–154)

Two ways to build a pipeline:

DIY pipeline	Opinionated platform
You write every script: build, test, deploy, rollback, log, monitor	Platform detects project type and builds the pipeline for you
Total control, huge upfront cost (weeks of scripting)	Zero config to start, standardized best-practice steps
Maintenance forever	Trade-off: less control over the pipeline itself

Popular opinionated CD platforms (most align with W4 standards — OCI, containers):

GitHub Actions · GitLab CI · Vercel · Render · Railway · Azure App Service · Cloud Run · Heroku

For Lab 8: GitHub Actions. Why: free for public repos, lives next to your code, YAML is human-readable. You'll push a Node.js or Python app, and a workflow will lint → test → deploy to a free PaaS.

7. Challenges to CI/CD adoption (Ch 11, p. 154)

Ch 11 is honest — tools are easy, culture is hard. Five common reasons teams fail:

#	Challenge	What it looks like in real life
1	Skills gap	Nobody on the team knows YAML, Docker, or the pipeline tool
2	Fear of automation	Ops worry: "if the script breaks prod, who gets blamed?"
3	Legacy apps	Monolithic code without tests = nothing safe to automate
4	Tool sprawl	10 disconnected tools (Jenkins, Jira, Slack, PagerDuty, ...) — nobody sees the whole picture
5	Incentive mismatch	Devs rewarded for features, ops rewarded for uptime — both lose when forced to share

Reality check: most "DevOps failures" are not technology — they are organizational. You can buy every tool and still ship badly.

8. Self-test checkpoints

- ☐ Name the 5 pillars of DevOps (CALMS).
- ☐ What did DevSecOps add on top of DevOps? When did security join the loop?

- ☐ Differentiate CI, Continuous Delivery, and Continuous Deployment in one sentence each.
- ☐ Why does "CD" cause so much confusion? What are the two meanings?
- ☐ Name the 5 stages of a typical pipeline in order.
- ☐ What is "opinionated CD"? Give one pro and one con vs a DIY pipeline.
- ☐ Why do microservices + containers make CI/CD easier than monoliths?
- ☐ Name 2 non-technical reasons CI/CD adoption fails.

9. Lab 8 — Build a CI/CD pipeline with GitHub Actions (3 hours)

Goal: Push a tiny app to GitHub. Watch tests run automatically. Get a public URL. Verify the same URL updates when you push again.

Time	Activity	Deliverable
0:00–0:15	Create a public GitHub repo: <code>w8-cicd-pipeline</code> · add a minimal Node.js (or Python) app + 1 unit test that passes	Repo with green test locally
0:15–0:30	Add <code>.github/workflows/ci.yml</code> · trigger: every push · job: install + test · push it	First Actions run (green)
0:30–0:45	Verify: on github.com → Actions tab → see the workflow ran · download the green checkmark screenshot	Screenshot: green run
0:45–1:00	Break the test on purpose (change a constant) · push · watch it fail · screenshot the red run · fix the test · screenshot the new green run	Screenshot: red run + recovery
1:00–1:30	Sign up on Vercel / Render / Railway · connect the GitHub repo · configure auto-deploy on push to <code>main</code>	Auto-deploy configured
1:30–1:50	Get the public URL · open on phone · make a small change in the app · push to <code>main</code> · verify the URL updates within 2 min	Screenshot: phone + URL
1:50–2:00	Write 1-paragraph reflection: "Which stage of the pipeline took the most time, and why? What would I automate next?"	Reflection
2:00–3:00	Self-study: add a second workflow file that runs ONLY on pull requests · try the same YAML for a Python (Flask) app · explore the GitHub Actions marketplace	Optional stretch

Minimal example app (Node.js)

package.json

```

{
  "name": "w8-cicd",
  "version": "1.0.0",
  "scripts": {
    "start": "node index.js",
    "test": "node test.js"
  },
  "engines": { "node": "20.x" }
}

```

index.js — a tiny HTTP server that returns build info:

```
const http = require('http')
const port = process.env.PORT || 3000
const build = process.env.BUILD_SHA || 'local'
http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/html' })
  res.end(<h1>CI/CD demo</h1><p>Build: ${build}</p>)
}).listen(port)
```

test.js — a single passing test (using Node's built-in `assert`):

```
const assert = require('assert')
assert.strictEqual(1 + 1, 2)
console.log('✓ all tests passed')
```

Minimal GitHub Actions workflow

`.github/workflows/ci.yml`

```
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npm test
```

Rubric (out of 10)

- Green Actions run visible on github.com — **20%**
- Red Actions run + recovery (you broke it on purpose) — **20%**
- Public URL works on desktop AND phone — **20%**
- URL updates automatically after a new push to `main` — **20%**
- Reflection paragraph (which stage took longest + next automation) — **20%**

10. What's next (W9 — Managing Workloads in a Multicloud Environment)

W9 zooms out again. Now that you can build + deploy + monitor a single app, we ask: **where should this app run?** Ch 12 covers workload placement, multicloud architecture, and the same 4-considerations framework (latency / security / governance / reliability) applied to "which cloud?".

Lab 9: design a 3-tier app's multicloud placement for a Thai e-commerce site on 11.11 (Singles' Day traffic). The point: the 11.11 traffic spike needs geographic + provider diversity, not just one giant VM.